基于解码器的语言模型实验报告

姓名 胡淳启　　班级 软件2404班　　学号 2246111996

2026年 10月 23日

一 实验内容

依照题目要求，编写一个基于解码器的语言模型，用前面的词来预测下一个词。多头注意力、因果掩码和归一化等部分自己编写，不能直接调用现成模块。先在四层小模型上进行训练和调试，再建立约300M参数的模型，完成参数统计和推理演示。

二 算法设计

数据处理

选用《爱丽丝梦游仙境》的英文文本作为数据集。先去掉正文前后的说明，将英文转为小写，统一引号，再用正则表达式将单词和标点分开。根据训练文本建立词表，给每个词元分配编号。词表共 2510 项，其中包含 <unk>、<bos> 和 <eos>，分别用来表示未知词、开始和结束。

按段落将前约 90% 的文本用于训练，后约 10% 用于验证。处理后训练集有 31,305 个词元，验证集有3,061 个词元，均包含开始和结束标记。词表只用训练集创建，验证集遇到没有出现过的词就记为<unk>，这部分约占验证集的 5.42%。

模型设计

先将词元编号转换成向量，加上位置向量后传入解码器。每层先进行归一化，再计算因果多头注意力并加上原来的输入；接着进行归一化和前馈计算，再加上上一部分的结果。最后把向量映射到词表大小，得到下一个词的分数。

小模型和大模型使用同一套类，通过 config.py 设置不同参数。具体设置如下。

参数	小模型	约 300M 模型
解码器层数 L	4	24
隐藏维度 D	128	1024
注意力头数 H	4	16
每头维度 D/H	32	64
前馈维度 F	512	4096
上下文长度 T	64	128
词表大小 V	2510	2510
位置编码	可学习位置嵌入	可学习位置嵌入

运行环境为 Windows、Python 3.12.9 和 PyTorch 2.5.1+cpu，使用 CPU 运行，线程数为 4。

三 主干代码说明

基础层与嵌入

layers.py 中创建 LinearMap 和 EmbeddingTable 两个类。LinearMap 定义 weight 和 bias，用矩阵乘法完成线性变换；EmbeddingTable 保存一张参数表，根据输入编号取出对应向量即可。这里没有调用 nn.Linear 和 nn.Embedding。

```
return values @ self.weight + self.bias
```

```
return self.weight[indices]
```

因果多头注意力

CausalAttention 中分别创建 query、key、value 三个映射来得到 Q、K、V。用 split_heads() 将 [B, T, D] 变为 [B, H, T, D/H]，使每个头分别进行计算。B 是批大小，T 是序列长度，D 是隐藏维度，H 是头数。

$$A = softmax\left(\frac{QK^T}{\sqrt{d}} + M\right), \qquad O = AV$$

其中 d 为每头维度。M 在当前位置和之前的位置取 0，在后面的位置取负无穷。对应的掩码代码如下，future 为 True 的位置就是不能读取的未来词元。

```
positions = torch.arange(length, device=values.device)
future = positions[None, :] > positions[:, None]
scores = scores.masked_fill(future, float("-inf"))
weights = stable_softmax(scores)
attended = weights @ contents
```

stable_softmax() 先减去每行的最大值，再计算指数并除以总和，避免数值过大。经过这一步，未来位置的权重为 0。最后将各个头的结果合并，再通过 output 映射返回。

归一化与前馈网络

LayerNormalization 中用 scale 和 offset 保存缩放及偏移参数。对每个词元的特征求均值和方差，再进行归一化。这里的方差除以 D，不是 D−1，epsilon 设为 10^{-5}。设均值为 μ，方差为 σ^2，计算方式如下。

$$N(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

FeedForward 先用 expand 将 D 维扩展为 F 维，按公式计算 GELU 的 tanh 近似，再用 contract 变回 D 维。注意力输出映射和 contract 的初始化标准差根据层数缩小，避免多层残差累加后数值过大。

主干代码说明 续

完整模型

model.py 中创建 DecoderLayer 类，里面分别定义 attention_norm、attention、feedforward_norm 和 feedforward。forward() 中依次调用，并将结果加回输入。两次相加就是残差连接。

```
states = states + self.attention(self.attention_norm(states))
states = states + self.feedforward(self.feedforward_norm(states))
```

DecoderLanguageModel 中定义 words、positions 和 decoder。words 保存词元嵌入，positions 保存位置嵌入，decoder 用来储存多层 DecoderLayer。输入经过这些层后，用 final_norm 进行最后一次归一化，再乘以 words.weight 的转置，得到 [B, T, V] 的输出。输出映射和词元嵌入使用同一份参数，统计时只计算一次。

参数统计

count_params.py 中用 count_parameters() 遍历模型参数，通过 numel() 取得每个参数张量的元素数量并累加。seen 用来记录已经计算过的参数，避免重复。另用 expected_parameters() 根据各层尺寸计算一次，比较两个结果是否相同。

一层解码器包括四个注意力映射、两个前馈映射和两个归一化。每层参数量及总参数量分别为：

$$P_{layer} = 4D^2 + 2DF + 9D + F$$

$$P_{total} = (V + T)D + LP_{layer} + 2D$$

部分	小模型	约 300M 模型
词元嵌入	321,280	2,570,240
位置嵌入	8,192	131,072
全部解码器层	793,088	302,309,376
末尾归一化	256	2,048
总参数量	1,122,816	305,012,736

统计程序使用 meta 设备，只建立参数形状，不分配全部权重；大模型推理时再创建带有真实权重的模型。实际统计与公式计算的结果相同。

训练和生成

train.py 中先读取数据并建立词表，再创建模型和 AdamW 优化器。每次取出一批连续文本，将输入和标签错开一位。例如输入是 alice was beginning，标签就是 was beginning to。用 next_token_loss() 计算交叉熵，然后清空梯度、反向传播、裁剪梯度，最后更新参数。

generate.py 中用 load_checkpoint() 读取配置、词表和权重。generate_tokens() 每次取最后一个位置的输出，选出下一个词元并接到原序列后面，再继续预测。序列超过上下文长度后，只保留最近的一段。

四 运行结果展示

小模型训练

先在四层小模型上训练，共进行 400 步。使用的训练参数如下。

训练参数	设置
批大小与序列长度	8，每段 64 个词元
优化器与学习率	AdamW，0.001
权重衰减	0.01
梯度裁剪	梯度范数上限 1.0
随机种子	2026
损失记录	每 50 步评估一次，各固定 8 批

训练时随机取样，记录曲线时使用固定的训练批次和验证批次，便于比较前后的变化。图中的验证损失只在这 8 批上计算，没有遍历整个验证集。

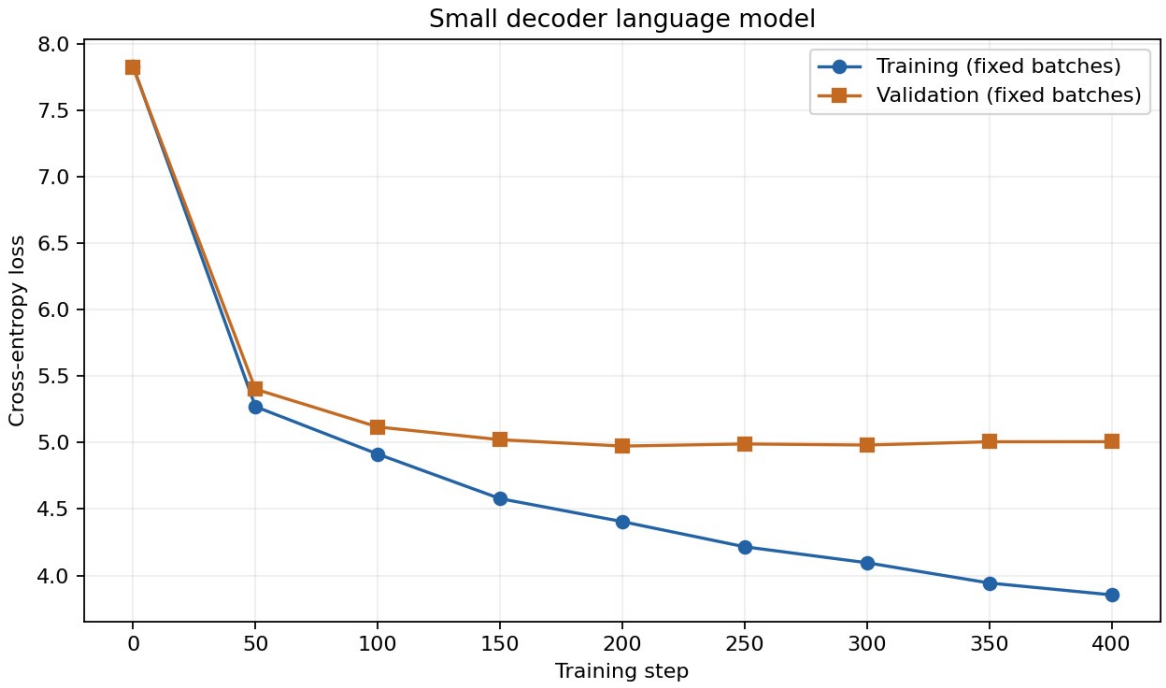


图 1 训练损失与验证损失

步数	训练损失	验证损失
0	7.8334	7.8236
200	4.4054	4.9738
400	3.8541	5.0068

从图中可以看出，前面一段下降比较明显。到 200 步左右，验证损失已经基本不再下降，之后还有小幅回升，训练损失则继续下降，有一定过拟合的情况。因此训练到 400 步就停止，没有继续增加步数。本次训练和中间评估共用时 37.925 秒，已经能看到损失下降，符合题目要求。

运行结果展示 续

字符串生成样例

使用第 400 步保存的模型，每次生成 40 个新词元。temperature 设为 0.8，top_k 设为 30，也就是从分数最高的 30 个候选中采样。以下为实际输出。

输入 alice was　随机种子 2026

alice was quite a few way to get out of its face again. ' she was at be a very large golden as she had been over to herself, as she thought, ' ' i think - - -

输入 the rabbit　随机种子 2027

the rabbit was not, and looked at the same, but the rabbit. ' and went on his arm, and was very very moment. ' but she went on alice turned to grow up from the little key

输入 she said　随机种子 2028

she said the time, and then the caterpillar. ' ' i don't see - - - - - i have been, ' ' of alice? ' ' alice had to herself it would you wouldn't be an very

生成的内容中有 was、she had 等常见搭配，也能看到 rabbit、caterpillar、key 这些文本里的词。但是整句话并不通顺，有些地方还会重复。小模型目前只是开始学到一些词语之间的联系，生成效果还比较有限。

约 300M 模型推理

将层数设为 24，隐藏维度设为 1024，得到 305,012,736 个参数。这里只做参数统计和推理，没有训练大模型，也没有把小模型权重放进去。创建真实的 float32 权重后，输入 alice was，检查一次前向输出，再生成 8 个词元。

项目	结果
参数量	305,012,736
参数存储	1163.531 MiB，不包含运行时其他内存
输出形状	[1, 2, 2510]，数值均为有限值
初始化耗时	3.271 秒
推理耗时	1.045 秒，含首次检查和后续生成

生成结果：alice was shedding struck beauti farther swallow would sentenced forgot

由于大模型的权重是随机初始化的，这组结果没有连贯的意思。这里主要检查参数量是否接近 300M，以及模型能否正常完成推理。

五 程序调试过程

输入和注意力的检查

先用 checks.py 检查各个部分。输入和标签需要错开一位，不能让模型预测当前已经输入的词。因此先比较一个窗口中的相邻输入和标签，确认对应关系正确。

注意力部分主要检查掩码。先查看未来位置的权重是否为 0，每行权重和是否为 1。再把序列后半部分改掉，比较前半部分的输出。前面的输出保持一致，说明没有用到后面的词。这个检查分别在注意力层和完整模型上进行。

数值和训练的检查

对 softmax、层归一化和交叉熵，分别与参考计算结果进行比较，结果一致。归一化还进行了双精度梯度检查，并检查常量输入会不会产生异常值。参考接口只在检查程序中使用，模型里的对应部分仍然是自己编写的。

反向传播后，逐个查看参数梯度，确认梯度存在且没有 NaN 等异常值。另外使用一组很小的固定样本训练 60 步，损失从 2.5564 降到 0.0079，说明前向计算、损失和参数更新可以正常配合。这里的数据仅用于调试，不能作为小说语料上的训练结果。

参数统计和保存加载

小模型和大模型的参数统计都与公式计算结果相同。分词器检查了大小写、引号和未知词处理。模型保存后重新加载，使用相同输入和随机种子生成，结果与保存前一致。超过上下文长度时也能继续生成，说明保留最近一段输入的方法可以正常运行。以上共 10 项检查均通过。

环境问题

最初安装的较新 PyTorch 在导入时出现 c10.dll 加载错误，改用 CPU 版 2.5.1 后可以正常运行。下载文本时主站连接失败，改用同一书籍的 GITenberg 镜像，原文和实际下载地址都保存在 data 文件夹中。

六 实验总结

本次实验完成了因果多头注意力、归一化和解码器语言模型的编写，能够进行下词预测训练和字符串生成。小模型训练后损失明显下降，约 300M 的模型也完成了参数统计和推理。当前主要的问题是生成内容不够连贯，验证损失到后面也不再下降，因此没有继续追求更好的效果。

附录 运行说明与数据来源

在 PowerShell 中进入 E:\NLPllm 后，用项目里的 Python 运行即可。重新训练时指定新的输出目录，避免覆盖已有结果。

```
.\.venv\Scripts\python.exe checks.py
.\.venv\Scripts\python.exe train.py --steps 400 --output runs/new_run
.\.venv\Scripts\python.exe generate.py --prompt "alice was"
.\.venv\Scripts\python.exe count_params.py --size 300m
.\.venv\Scripts\python.exe demo_large.py --new-tokens 8
```

文本来源为 Project Gutenberg 的第 11 号电子书，网址为 https://www.gutenberg.org/ebooks/11。实际下载地址及原文校验值见 data/source.json，实验日志和生成记录保存在 runs 文件夹中。